



Tengo el siguiente código de Arduino IDE. Necesito que según el valor resultante de la función `poteFiltrado()`, pueda manipularse la velocidad del texto.



Debes modificar por completo la estructura de la función `void loop()` .

El problema con tu estructura actual es que los ciclos `for` que reproducen el poema son "bloqueantes". Al usar esos ciclos, el Arduino lee el potenciómetro una sola vez, entra a los `for` , y no vuelve a leer el potenciómetro hasta que el poema termina de mostrarse completo.

Para que la velocidad y dirección cambien en tiempo real, debemos eliminar los ciclos `for` anidados y usar variables estáticas que recuerden en qué verso (`v`) y en qué letra (`pos`) está la pantalla en cada ciclo del `loop()` , leyendo el potenciómetro antes de cada movimiento.

Reemplaza todo tu `void loop()` con el siguiente código:

C++



```
void loop() {  
  // Variables estáticas para recordar la posición entre cada ciclo del loop  
  static int v = 0;  
  static int pos = 0;  
  
  // ----- lectura pote -----  
  poteLectura = analogRead(potePatita);  
  poteFiltrado = filtrarConDivision(poteLectura, 4);  
  
  Serial.print("valor filtrado ");  
  Serial.println(poteFiltrado);  
  // ----- fin lectura pote -----  
  
  // 1. Determinar dirección y calcular velocidad (pausa del delay)  
  int direccion = 0;  
  int pausa = 0;  
  
  if (poteFiltrado > 127) {
```

```

    direccion = 1; // Avanzar
    // map() convierte el rango del pote (128 a 255) a un delay en milisegur
    // Mientras más cerca del 255, menor es el delay (más rápido)
    pausa = map(poteFiltrado, 128, 255, 600, 50);
} else {
    direccion = -1; // Retroceder
    // Mientras más cerca del 0, menor es el delay (más rápido)
    pausa = map(poteFiltrado, 127, 0, 600, 50);
}

// 2. Cálculos del verso actual
int totalVersos = sizeof(versosPoema) / sizeof(versosPoema[0]);
int largoVerso = strlen(versosPoema[v]);
int pasosTotales = (largoVerso > 16) ? (largoVerso - 16 + 3) : 0;

// 3. Imprimir el texto en la pantalla
lcd.setCursor(0, 0);
for (int i = 0; i < 16; i++) {
    if (pos + i < largoVerso && pos + i >= 0) {
        lcd.print(versosPoema[v][pos + i]);
    } else {
        lcd.print(' ');
    }
}

// 4. Aplicar la velocidad calculada
delay(pausa);

// 5. Actualizar la posición para el siguiente ciclo
pos += direccion;

// 6. Lógica para cambiar de verso si llegamos al límite (avanzando o retr
if (pos > pasosTotales) {
    // Si avanza más allá del verso actual, pasa al siguiente
    pos = 0;
    v++;
    if (v >= totalVersos) v = 0; // Vuelve al inicio si terminó el poema
    lcd.clear();
    delay(800); // Pausa visual al cambiar de línea
}
else if (pos < 0) {
    // Si retrocede más allá del inicio, vuelve al verso anterior
    v--;
    if (v < 0) v = totalVersos - 1; // Va al último verso si retrocede desde

    // Recalcula el tamaño del nuevo verso para posicionarse al final de est
    largoVerso = strlen(versosPoema[v]);
    pasosTotales = (largoVerso > 16) ? (largoVerso - 16 + 3) : 0;
    pos = pasosTotales;
}

```

```

    lcd.clear();
    delay(800);
  }
}

```

La función `map()` es la clave de esta lógica. Toma el rango dinámico del potenciómetro (por ejemplo, de 128 a 255) y lo convierte proporcionalmente a un rango de tiempo en milisegundos (de 600 a 50). Puedes ajustar esos dos últimos números de la función `map()` si sientes que el texto se mueve demasiado lento en su punto base o demasiado rápido en su punto máximo.

menciona que diferencias existen con el anterior, además de como implementarlos

Aquí tienes las diferencias principales entre tu código original y el nuevo, explicadas paso a paso, seguidas de las instrucciones sobre cómo implementarlo en tu Arduino IDE.

Diferencias principales con el código anterior

1. Eliminación de los ciclos `for` (De bloqueante a continuo):

El tuyo: Usaba ciclos `for` anidados para reproducir el poema. Esto obligaba al Arduino a terminar todo el poema antes de poder volver a leer el valor del potenciómetro al inicio del `loop`.

El nuevo: Se deshace de los `for`. Al dejar que la función `loop()` haga su ciclo natural de forma continua, el Arduino ahora lee el potenciómetro **antes de mover cada letra**, permitiendo que los cambios respondan en tiempo real.

2. Uso de variables `static`:

El tuyo: Declaraba `int v = 0;` e `int pos = 0;` dentro de los ciclos `for`. Al terminar, esas variables se destruían.

El nuevo: Declara `static int v = 0;` y `static int pos = 0;`. La palabra `static` significa que, aunque el `loop()` termine y vuelva a

empezar, el Arduino "recuerda" en qué verso y en qué posición de la letra se quedó en el ciclo anterior.

3. Control de Dirección (Avanzar o Retroceder):

El tuyo: Siempre sumaba hacia adelante (`pos++`).

El nuevo: Crea la variable `direccion` . Si el valor es mayor a 127, `direccion = 1` (avanza). Si es menor o igual a 127, `direccion = -1` (retrocede). Al final del ciclo, se aplica matemática básica:
`pos += direccion; .`

4. Velocidad dinámica con la función `map()` :

El tuyo: Usaba tiempos fijos estáticos (`delay(1500)` y `delay(300)`).

El nuevo: Utiliza la función matemática `map()` . Esta función toma un rango de valores (ej. 128 a 255) y lo transforma proporcionalmente a un rango de tiempo en milisegundos (ej. 600ms a 50ms). Mientras más giras la perilla alejándote del 127, el `delay` se hace más pequeño, haciendo que el texto "corra" más rápido.

5. Lógica de transición de los versos:

El tuyo: Como avanzaba siempre hacia adelante, el límite lo marcaba el fin del ciclo `for` .

El nuevo: Se añaden condiciones `if` al final. Si la posición sobrepasa el límite del verso hacia la derecha (`pos > pasosTotales`), salta al verso siguiente. Si retrocede antes de la primera letra (`pos < 0`), salta al verso anterior y se posiciona al final de dicha frase para seguir retrocediendo.

Cómo implementarlo en tu Arduino IDE

La implementación es un simple reemplazo de texto, ya que la estructura global de tu programa (las variables iniciales y el `setup()`) está perfecta.

1. Abre tu código en **Arduino IDE**.
2. Desplázate hacia abajo hasta encontrar la línea que dice `void loop() {` .
3. **Borra absolutamente todo** desde esa línea hasta la llave de cierre `}` que marca el final de tu documento.
4. **Copia** el bloque de código completo del `void loop()` que te dejé en la respuesta anterior.
5. **Pégalo** exactamente donde borraste el anterior.
6. Haz clic en el botón de **Verificar** (el ícono de la palomita/ticket en la esquina superior izquierda) para asegurarte de que no haya errores de sintaxis al copiar.

7. Conecta tu placa Arduino, asegúrate de tener seleccionado el puerto correcto y haz clic en **Subir** (el ícono de la flecha apuntando a la derecha).

¿Quieres que te explique cómo ajustar los tiempos mínimos y máximos en la función `map` por si sientes que el texto se mueve muy rápido o muy lento?